

Title: Emergency Medical Dispatch

Brief Description

This project simulates an ambulance dispatch system for a city. When emergency calls arrive, the system assigns the nearest available ambulance using a **Graph** for routing, prioritizes calls using a **Priority Queue** (by medical severity), and queues non-urgent calls using a regular **Queue** when all ambulances are busy.

Problem Statement

During emergencies, dispatchers waste critical time manually locating ambulances and estimating routes. A system that combines graph-based pathfinding, severity-based prioritization, and fair handling of non-urgent calls can reduce response times and save lives.

Scenario

A dispatcher receives three calls in quick succession:

1. **Heart attack** (Severity 1 – highest) at Location A
2. **Minor car accident** (Severity 3 – lowest) at Location B
3. **House fire with burns** (Severity 2 – medium) at Location C

System behavior:

- The **Priority Queue** automatically orders calls as: Heart attack → Fire → Car accident.
- The system checks available ambulances via the **Graph**, finds the closest ambulance to the heart attack, and computes shortest route (Dijkstra).
- If both ambulances are busy, the fire call waits in the **Priority Queue** and the car accident waits in the regular **Queue**.
- When an ambulance becomes free, the system checks the **Priority Queue** first. If empty, it takes the next call from the regular **Queue**.

Marking Rubric (Total = 15 Marks)

Criteria	Excellent (3)	Good (2)	Satisfactory (1)	Poor (0)	Marks
Graph Implementation & Pathfinding	Graph correctly built (nodes/edges/weights). Dijkstra works flawlessly for shortest path ETA.	Graph works. Dijkstra works but minor bug.	Graph incomplete or Dijkstra only works for simple cases.	No graph or no pathfinding.	3
Priority Queue (by severity)	Correct max-heap behavior. Severity 1 always processed before 2 and 3. Insertion/removal $O(\log n)$.	Works but severity order has one mistake.	Priority exists but not fully correct.	No priority queue or completely wrong.	3

Queue (FIFO for non-urgent calls)	Correct FIFO. Non-urgent calls processed only when priority queue empty. Works with dynamic ambulance availability.	FIFO works but integration with priority queue has minor flaw.	Queue works standalone but not integrated.	No queue.	3
System Integration & Workflow	Calls flow correctly: Priority Queue → assign ambulance → route → free ambulance → next call from PQ or regular Queue.	Major flow works but one step broken.	Partial integration.	No integration.	2
Code Quality & Modularity	Clean, well-commented, separate files/modules for each DS.	Good but some duplication.	Messy but readable.	Unreadable.	2
Demonstration & Report	Clear demo with test cases. Explains why Graph, PQ, Queue were chosen. Complexity analysis included.	Demo works. Missing complexity analysis.	Minimal demo or report.	No demo.	2

Deliverables:

1. **Source code** (Java) with:
 - Graph class (Dijkstra method)
 - Priority Queue class (or use heap)
 - Queue class (or use deque)
 - Main simulation loop
2. **Report** (2–3 pages)
3. **Demo video** (5 minutes)